GWDGUI
User Manual

Configure engines, play games, analyze positions, run tournaments, and manage PDN databases.

First edition
Generated from the current project manual

Contents

1. Getting started 3

2. Configuring engines 4

3. Starting and playing a game 7

4. Browsing games and variations 8

5. Analysis 9

6. Scores, score history, and evaluation bar 10

7. Running a tournament 11

8. PDN and FEN 13

9. Game database 14

10. Preferences 15

11. Common problems 16

12. Installation and developer notes 17

This is a first-version manual for the current GWDGUI application. GWDGUI is a graphical workbench for international draughts. It can play and browse games, run Hub and DXP engines, organize engine tournaments, display engine analysis, and store or search games in an SQLite database.

1. Getting started

When GWDGUI starts, the main window shows:

- the main draughts board and clocks;
- **Game History** and the legal moves for the displayed position;
- player names, result, FEN, and the game notation;
- the engine and annotator analysis areas;
- a score-history graph and evaluation bar;
- buttons for **Engines...**, **New game...**, **Tournament...**, analysis time, and **Auto Play**.

Before playing against an engine or running a tournament, register the engine through **Engines...**. Human-versus-human games can be started immediately.

The standard international-draughts starting position is:

```
W:W31-50:B1-20
```

2. Configuring engines

Open **Engines...** from the toolbar. The engine list shows each executable and its Hub or DXP identity.

Right-click the list to:

- **Add engine**;
- **Delete engine**.

Double-click an existing entry to edit it. Engine identities must be unique within the same protocol. DXP engines must not be configured to use the same local port at the same time.

2.1 Hub engines

In the engine editor:

1. Select the engine executable with **Browse**.
2. Set **Type** to `hub`.
3. Enter the Hub ID and any required launch arguments.
4. Click **Launch engine**.
5. Review the startup log and parameters reported by the engine.
6. Click **Launched correctly** to save the registration and parameter values.

GWDGUI starts the executable in the executable's own directory, initiates the Hub protocol, applies the configured non-GUI parameters, and initializes the engine. Closing the editor stops an engine that was launched for this test.

Hub engines can be used as players and annotators. A Hub engine can also be offered as an MCTS annotator when `gui-engine-supports-mcts` is enabled.

2.2 DXP engines

For a DXP engine:

1. Select the executable.
2. Set **Type** to `DXP`.
3. Enter a DXP ID.
4. Set the host, normally `127.0.0.1` for a local engine.
5. Set the port, commonly `27531` unless the engine requires another port.
6. Choose the role:

- `listen`: GWDGUI waits for the DXP peer to connect;
- `connect`: GWDGUI connects to the configured host and port.

1. Enter any required launch arguments.
2. Click **Launch engine** and inspect the connection log.
3. Click **Launched correctly** after the test succeeds.

Use a separate port for each simultaneously active DXP connection. Enable `gui-dxp-supports-fischer` only when the engine really supports Fischer increment through DXP.

DXP is used for game communication. DXP game messages do not currently supply the Hub-style analysis score and PV data used by the score history.

2.3 Launch arguments and configuration files

An engine may have an associated INI or configuration file. The editor can open and edit its contents. The following placeholders are expanded in launch arguments and supported configuration text:

- `{ip}` or `{host}`: the configured host;
- `{port}`: the configured port.

This is useful for engines that need their socket settings in a command-line argument or generated configuration file.

2.4 Engine parameters

Parameters reported by a Hub engine are shown in the parameter grid. Their values are sent to the engine when it is initialized, except for names starting with `gui-`. Those are GWDGUI settings and are never sent as Hub parameters.

Important GUI parameters include:

Parameter	Meaning
`gui-engine-type`	`hub` or `dxp`
`gui-hub-id`	Hub identity used by GWDGUI
`gui-hub-launch-argument`	Hub command-line arguments
`gui-dxp-id`	DXP identity used by GWDGUI
`gui-dxp-ip`	DXP host or IP address
`gui-dxp-socket`	DXP port
`gui-dxp-role`	`listen` or `connect`
`gui-dxp-launch-arguments`	DXP command-line arguments
`gui-dxp-supports-fischer`	Whether the DXP engine supports increment
`gui-ini-file`	Optional engine configuration file
`gui-send-starting-position`	Whether the starting position is sent explicitly
`gui-single-captures-include-captured-square`	Retained move-format compatibility setting
`gui-engine-supports-mcts`	Makes a Hub engine available as an MCTS annotator
`gui-score-perspective`	Perspective of scores reported by a Hub engine
`gui-evaluation-bar-depth-min`	Retained engine-specific evaluation setting
`gui-evaluation-bar-score-max`	Retained engine-specific evaluation setting

`gui-score-perspective` tells GWDGUI how to convert an engine score to White's perspective:

- `side-to-move`: a score is good for the side to move; scores are negated when Black is to move;
- `white`: the reported score is already from White's perspective;
- `black`: the reported score is from Black's perspective and is negated.

Choose this setting to match the engine's protocol output. An incorrect choice causes the evaluation graph to change sign on the wrong moves.

The current score graph takes its display maximum and scale from **File > Preferences**. The seeded per-engine `gui-evaluation-bar-*` values are retained in parameter files for compatibility but are not the current graph controls.

2.5 Where engine settings are stored

GWDGUI stores the engine registry in `engines.json` beside the GWDGUI executable. Parameter values for an engine are stored beside that engine, using the engine executable name with the extension `.params.json`.

Keep copies of these files when moving an installation or maintaining several engine configurations.

3. Starting and playing a game

Click **New game...**. Configure:

- the starting FEN;
- White and Black as **Human** or a registered engine;
- moves per period, default `75`;
- minutes per period, default `5`;
- increment in seconds, default `0`;
- increment mode, **From start** or **After move limit**.

The same engine instance cannot play both sides, and an engine that is already busy is not available. Click the dialog's start button to create the game.

Engine startup and initialization happen before the game clock is charged. The playing clock starts when the game itself starts, so engine load time is also excluded in tournament games.

3.1 Making a human move

Click the source square and then the destination square. GWDGUI accepts only a legal move for the displayed position. You can also click a move in the **Legal moves** list.

When a capture has multiple landing squares or paths, use the legal-move list to select the exact move if the board clicks are ambiguous.

Press `F` to flip the board.

3.2 Managing games

The **Game History** list can hold multiple live, stopped, and finished games. Right-click a game to use the available actions:

- **New game...**;
- **Replay game**;
- **Show stdout**;
- **Stop game**;
- **Remove game**.

Replay starts another game with the same players and setup when the required engines are available. Stopping a game makes it available for browsing and analysis features such as Auto Play.

Selecting a running game follows its current position. Selecting a stopped or finished game lets you inspect its game tree and move history.

4. Browsing games and variations

Click a move in the notation to display the position after that move. Place the caret before the first move to display the starting position.

While the move list has focus:

- `Left Arrow` moves to the previous game position;
- `Right Arrow` moves to the next game position;
- `Escape` returns to the live or final game position.

Engine PVs, annotator PVs, and imported PDN variations are displayed between parentheses with move numbers. Clicking a move in a variation enters variation browsing. The arrow keys then cycle through all variation positions, including the position before its first move. Press `Escape` to leave variation browsing.

Analysis may continue for the position being browsed, but browsing an engine or annotator PV does not add its moves to the structural game tree.

4.1 Move-list display options

Right-click the move notation to show or hide:

- comments and annotations;
- engine scores;
- annotator scores;
- the engine PV move or full PV;
- the annotator PV move or full PV;
- clock information;
- the Game Tree window.

If both full PV options are enabled, the two variations can appear next to each other. They remain separate analysis lines.

4.2 The Game Tree window

The Game Tree window exposes the game tree that owns the current game state. Nodes can show engine scores and PVs, annotator scores and PVs, Auto Play PVs, and clock information.

Click a node to inspect its position. While a node is selected, automatic tree following is suspended; press `Escape` to resume following the current game. The tree window's context menu can copy the tree as PDN.

5. Analysis

5.1 Loading an annotator

Right-click the annotator analysis area and choose a registered Hub engine from the **Open Annotator** submenu. Use **Close Annotator** to stop it.

The annotator analyzes the position currently displayed on the main board, including a position selected while browsing. Its score and PV are stored as analysis attached to the relevant game-tree node.

Only Hub engines are available as annotators. An annotator engine is reserved while loaded and therefore cannot simultaneously be selected for an unrelated game that requires the same engine instance.

5.2 Analysis time

The edit box between **Tournament...** and **Auto Play** contains the analysis time in seconds. Its default is `1`. This value is used for timed annotator requests and Auto Play searches. A missing, invalid, or non-positive value is treated as one second.

5.3 Engine and annotator PVs

The engine PV belongs to the engine playing the game. The annotator PV belongs to the separately loaded annotator. Both are attached to their source nodes and can be displayed as browsable PDN variations.

Right-clicking the main board offers hint actions based on the main engine PV or annotator PV when those lines are available.

5.4 MCTS annotator

Right-click the MCTS analysis area to open or close an MCTS annotator. The selection lists Hub engines whose `gui-engine-supports-mcts` parameter is true. Statistics reported by that engine are displayed in the MCTS panel.

5.5 Auto Play

Auto Play is enabled only when:

- the selected game is stopped; and
- an annotator engine is loaded.

Auto Play starts from the currently selected position, asks the annotator for a best move using the configured analysis time, plays that move virtually, and repeats. Its moves are stored as an **Auto Play PV** attached to the source game-tree node. They are analysis, not new mainline game-tree nodes.

Press `Escape` to stop Auto Play. The moves already found remain in the Auto Play PV, but no later engine reply is appended. If the Auto Play PV is being browsed, press `Escape` again to leave variation browsing.

Leave any other PV-browsing mode before starting Auto Play.

6. Scores, score history, and evaluation bar

All scores displayed by the score history and evaluation bar are normalized to White's perspective:

- a positive score favors White;
- a negative score favors Black;
- zero is approximately equal.

The graph can contain both the playing engine's score and the annotator score. For each played ply, the annotator score attached to that node is used; when it is absent, the preceding node's annotator score can supply that point.

The evaluation bar uses the latest playing-engine score at or before the currently viewed ply. If no such engine score exists, it uses the latest annotator score. White occupies the lower part of the bar and Black the upper part. A large positive score makes White fill upward from below; a large negative score makes Black fill downward from above.

Hover over the score history to inspect the score at a ply. Right-click it to open the larger score-history window.

The graph's maximum score and **Linear** or **Logarithmic** scale are configured under **File > Preferences**. These are display settings; they do not change the score received from an engine.

7. Running a tournament

Click **Tournament...**.

7.1 Tournament settings

Enter:

- a tournament name;
- a starting FEN;
- moves per period;
- time in minutes;
- increment in seconds;
- increment mode;
- maximum concurrent games;
- pairing type;
- at least two engines from the checklist.

Available pairing types are:

- **Single round-robin**: each pair meets once;
- **Double round-robin**: each pair meets twice with reversed colors;
- **Swiss**: one round is generated at a time from the current standings.

Click **Create pairing**. Single and double round-robin creation builds the complete schedule. For Swiss tournaments, all results in the current round must be known before the next round can be created. GWDGUI warns when avoiding a repeat pairing is impossible.

Changing accepted tournament settings after creating pairings invalidates the schedule. Choose **Apply change** to clear the existing pairings and results, or **Revert** to restore the accepted settings.

7.2 Playing the tournament

Click **Play** to start pending games. GWDGUI starts games up to the configured concurrency limit while respecting engine availability. Each simultaneous game needs its own available engine instances and non-conflicting DXP resources.

The pairing grid shows game number, round, White, Black, result, and reason. Click a running row to display and follow that game's current tree in the main window. Finished games can be selected and browsed in the same way as other completed games.

Click **Stop** to stop the tournament's active games. Incomplete games remain available for review or rescheduling.

7.3 Results and rescheduling

Right-click a pairing to set its result manually to:

- `*` for unknown;
- `2-0` for a White win;
- `1-1` for a draw;
- `0-2` for a Black win.

A manually entered result is marked with the reason **Manual result**. Use **Reschedule selected game** to clear a game's result and make it pending again.

The cross table ranks engines by points, wins, Sonnenborn-Berger score, and then name. It also shows the individual matchup results.

7.4 Saving and loading tournaments

Use **Save** to write the tournament state as JSON. The file contains the settings, selected engine names, pairings, results, reasons, and concurrency limit. Use **Load** to resume it later. Loading is available only while the tournament is not running, and its engines must still be registered under the saved names.

Completed games with moves are also appended as PDN to a file named after the tournament. When the tournament JSON has been saved, the PDN is placed in the same directory; otherwise it is written in the current working directory.

The JSON is the resumable tournament administration file. The PDN is the game record. Preserve both. GWDGUI prompts to save tournament state when closing if unsaved tournament data exists.

8. PDN and FEN

8.1 Opening PDN

Choose **File > Open PDN**. GWDGUI lists the games in the file with White, Black, event, and result. Double-click a game to load it into the main window.

The loader reconstructs the starting position, mainline, and PDN variations in the game tree. Invalid moves are reported with their ply so a damaged game can be located.

8.2 Saving PDN

Choose **File > Save PDN** to save the displayed game tree. GWDGUI appends the game to the selected PDN file; it does not replace games already in that file. Choose a new filename when a one-game file is required.

The Game Tree window can also copy the current tree as PDN to the clipboard.

8.3 Working with FEN

The FEN field shows the displayed position. Right-click the main board to:

- copy its FEN;
- open **Setup position**;
- search that position in an open database.

In **Setup position**, select a piece type and click board squares to place or remove it. Choose the side to move. **Start** restores the normal starting position, **Clear** empties the board, and the FEN action parses the text entered in the FEN box. **Apply** sends the resulting position back to the main window; **Cancel** leaves the current position unchanged.

9. Game database

GWDGUI uses SQLite databases for collections of PDN games and position search.

Choose **Database > Create database** for a new `.sqlite` or `.db` file, or **Database > Open database** for an existing one.

9.1 Importing games

In the database window, right-click and choose **Import PDN**. The progress view shows imported games, errors, and duplicates, and allows the import to be stopped. An `.import.log` file is written beside the imported PDN file.

Use **Rebuild position summary** only when the position index needs to be recreated, for example after repairing or migrating a database.

9.2 Finding games

The game filters include White, Black, Event, Player, and a maximum number of rows. Click **Apply filter** after changing them. Click the White, Black, or Event column heading to sort; click it again to reverse the order. Double-click a game to load it in the main window.

9.3 Searching for a position

Enter or paste a FEN in the position panel and click **Search**. The result lists continuation moves with game, win, draw, and loss counts and provides matching games where practical.

The database board's context menu can copy or paste FEN and open the position setup window.

When a database is open, enable **Search position in database** from the main board's context menu. The database search then follows the position displayed while you play or browse.

10. Preferences

Choose **File > Preferences**.

The appearance settings control:

- light and dark squares of the main board;
- light and dark squares of analysis boards;
- target, last-move, PV-move, and hint highlighting.

The evaluation settings control:

- maximum displayed score;
- linear or logarithmic score scaling.

Use **Defaults** to restore default values, **Save** to apply and persist the settings, or **Close** to leave without saving changes. Preferences are stored in `prefs.json` beside the GWDGUI executable.

11. Common problems

An engine does not appear in a game dialog

The engine may already be reserved by another game or loaded as an annotator. Stop the other use or register a separate engine instance. Also check that the engine test was completed with **Launched correctly**.

A DXP engine cannot connect

Check the `listen`/`connect` role, host, and port. Confirm that the peer uses the opposite role and the same port, and that no other configured DXP engine is using that local port.

Scores have the wrong sign

Edit the Hub engine and set `gui-score-perspective` to match what the engine reports: side to move, White, or Black. GWDGUI's display itself always uses White's perspective.

The score history is empty

DXP games do not currently provide the analysis score stream used by this display. For Hub engines, confirm that the engine reports score information and that `gui-score-perspective` matches its output. Loading a Hub annotator can provide a separate annotator score.

Auto Play is disabled

Stop the selected game and load a Hub annotator. Auto Play is intentionally not available for a running game or without an annotator.

A tournament will not start all requested games

The concurrency value is an upper limit. Actual concurrency is also restricted by available engine instances and DXP port conflicts.

A saved PDN contains more than one game

Save PDN appends. Select a new file when you do not want to add the current game to an existing collection.

12. Installation and developer notes

GWDGUI is an Object Pascal application built with Free Pascal and Lazarus/LCL. The `draughts.ttf` piece font must be available beside the application as expected by the build or distribution.

A typical source build is:

```
lazbuild --build-mode=Default gwdgui.lpi
```

The repository's regression checks can be run with:

```
bash tests/run_regression_checks.sh
```

For protocol details, see DXP Protocol.md and protocol.txt in the source tree.